

计算机图形学原理

-----教程8

李瑞辉

大纲

- **Open Asset Import Library**加载模型
- 阴影贴图

Assimp

- 一个非常流行的模型导入库叫做Assimp，代表开放资源导入库

The Open-Asset-Importer-Lib

Main Menu

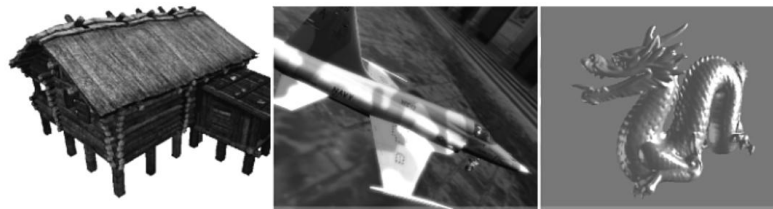
[Home](#)
[Features](#)
[Downloads](#)
[Blog](#)
[Docs](#)
[Viewer](#)
[Contact](#)
[License](#)
[Github-Page](#)
[Donate](#)

Become my patron on




Home

 Veröffentlicht: 16. Januar 2018
 Zugriffe: 166878

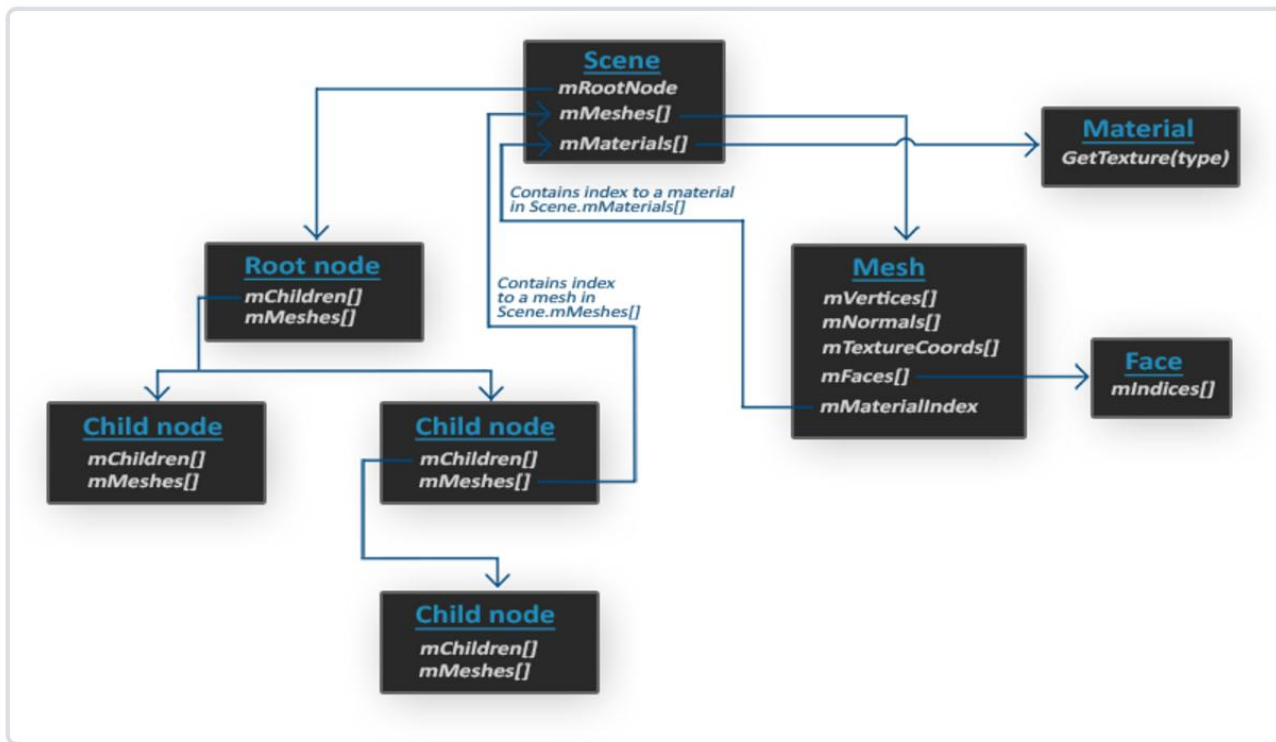


Open Asset Import Library (short name: Assimp) is a portable Open Source library to import various well-known [3D model formats](#) in a uniform manner. The most recent version also knows how to export 3d files and is therefore suitable as a general-purpose 3D model converter. See the [feature-list](#).

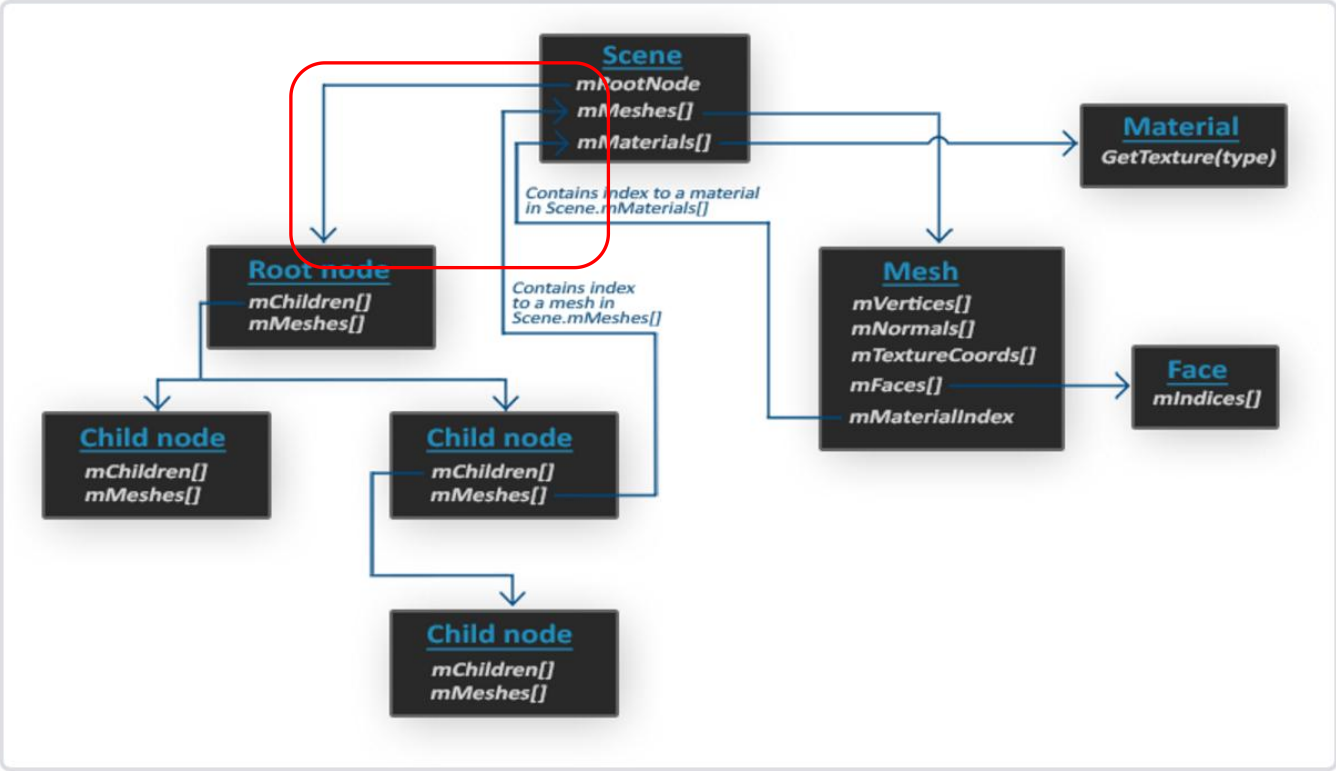
open3mod is a Windows-based [model viewer](#). It loads all file formats that Assimp supports and is perfectly suited to quickly inspect 3d assets.

Assimp

- Assimp中，Assimp能够导入十种不同的模型文件格式通用数据结构。
(如obj、ply、 stl)
- Assimp 的数据结构中检查我们需要的所有数据。



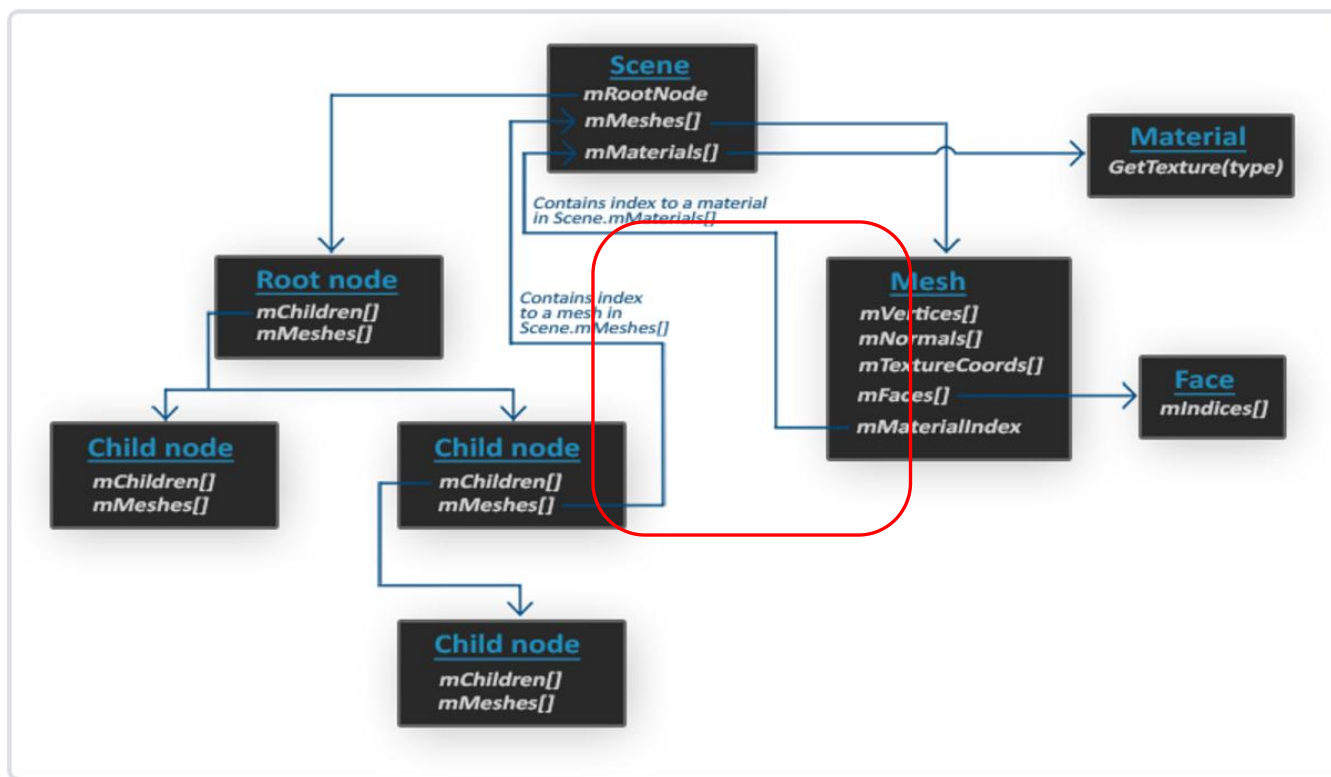
Assimp 的数据结构



Assimp 的数据结构

Assimp

Mesh对象本体包含渲染所需要的所有相关数据，想顶部位置、法向量、纹理坐标、面和对象的材质。



Assimp 的数据结构

Assimp

- 我们使用Assimp时的流程
 - 将对象加载到场景对象中
 - 从每个节点发送归查相关的Mesh 对象
 - 处理每个Mesh对象以检查顶部数据、索引及其材料属性。
 - 结果是我们想要包含在单个模型对象中的网络数据集。

Assimp

➤ 细节

- 构造Assimp

<https://learnopengl.com/Model-Loading/Assimp>

- 网格介绍

<http://learnopengl.com/Model-Loading/Mesh>

- 加载模型

<http://learnopengl.com/Model-Loading/Model>

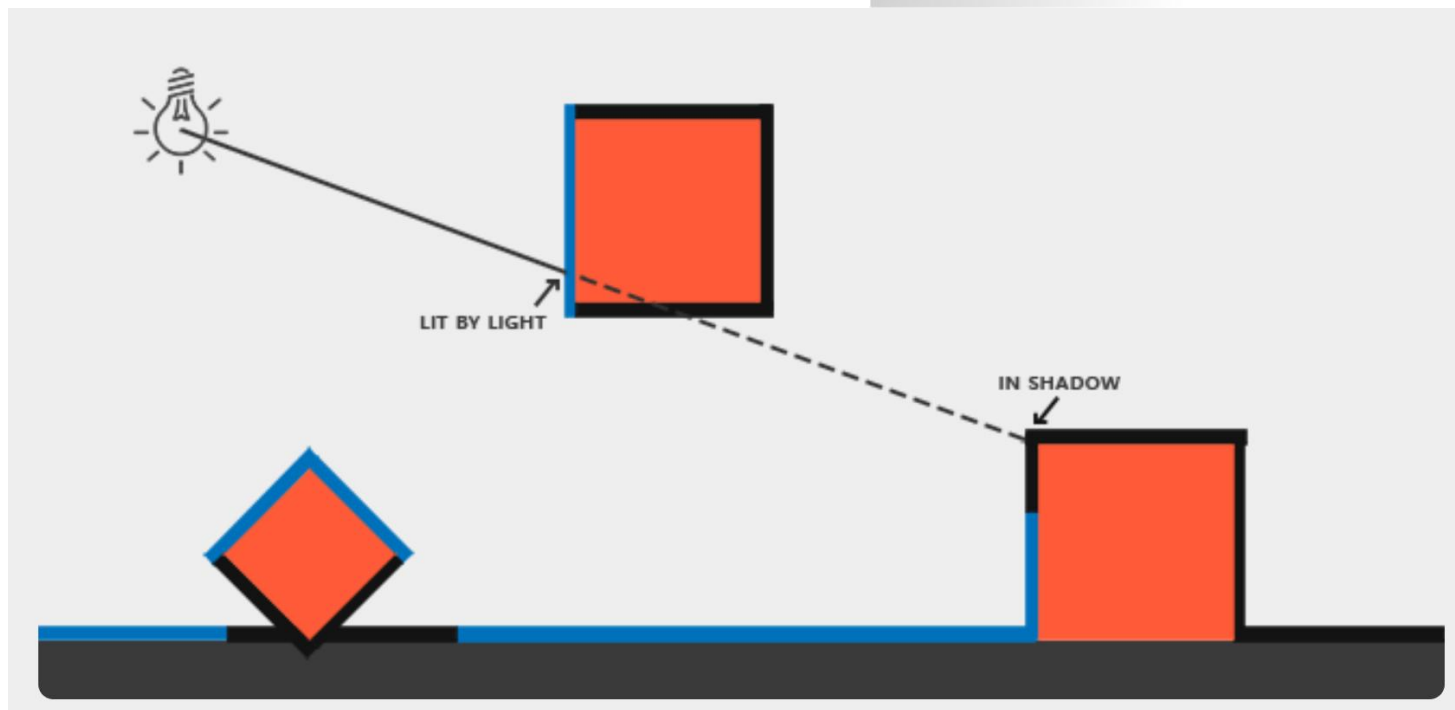
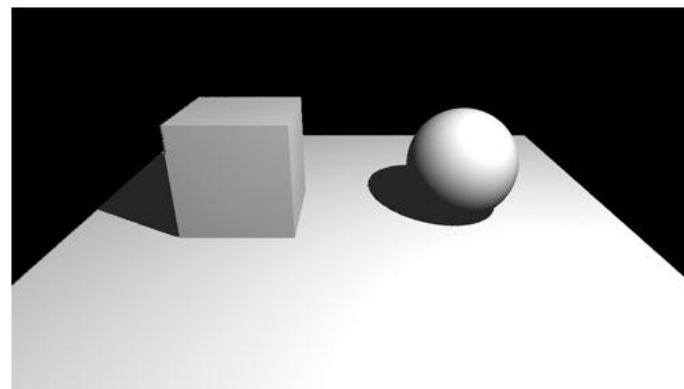
大纲

- 使用Open Asset Import Library加载模型
- 阴影贴图

阴影

➤什么是阴影

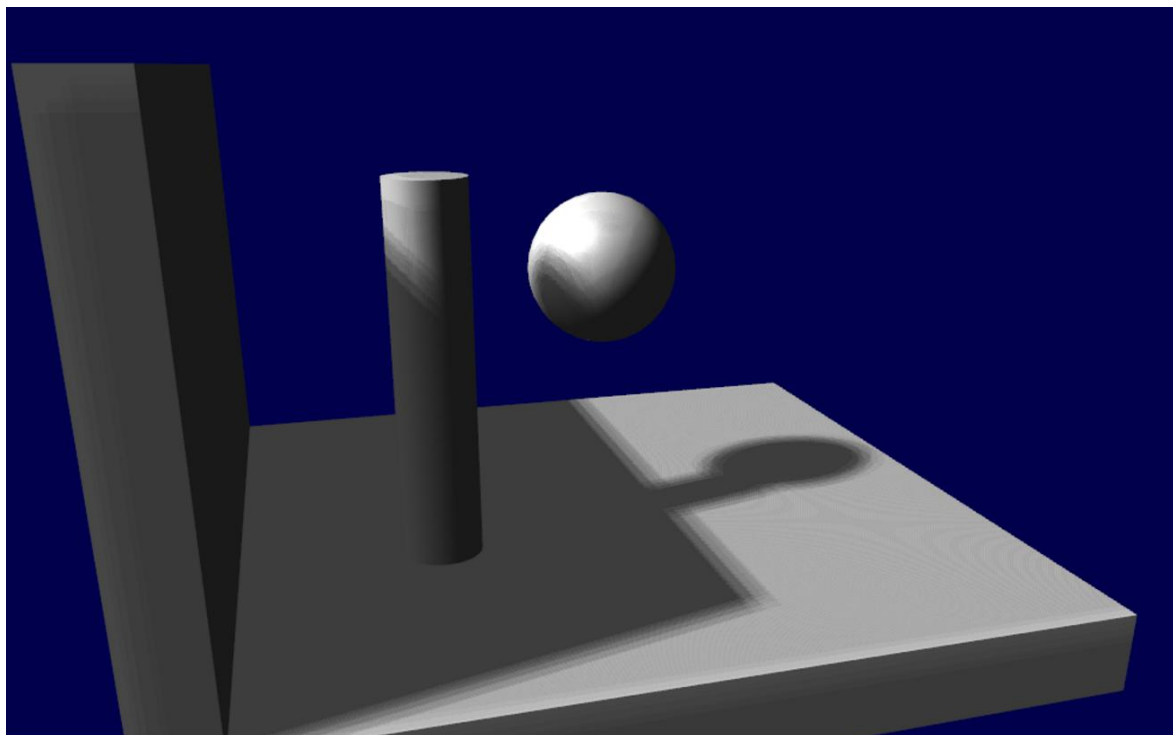
- 光不能通过



阴影贴图

➤ 阴影贴图

- 动态阴影
- 表演视频: https://www.youtube.com/watch?v=XF30cLr_-V8

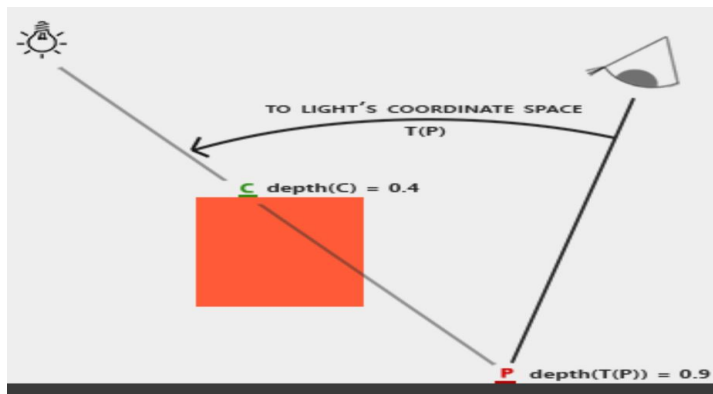


阴影贴图

➤ 算术[两次通过]

1. 渲染阴影贴图

- 场景是从光线的角度冲洗的
- 只计算每个查看片段的深度



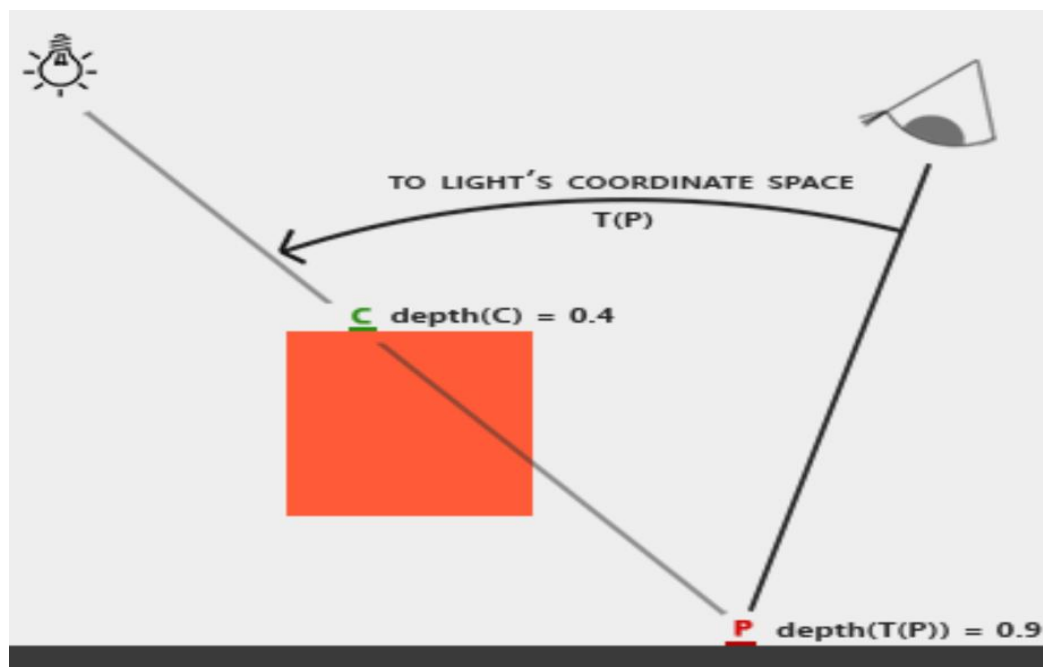
2. 经常使用阴影贴图渲染场景

- 场面像往常一样渲染，但有一个正面的测试来查看当前片断是否在阴影中。

阴影贴图

➤ 额外测试

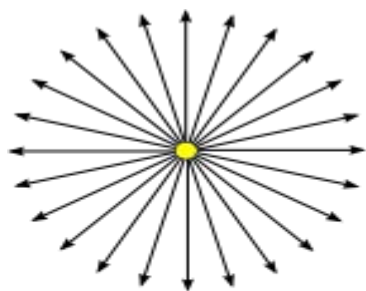
- 如果当时之前样本在同一点比阴影贴图离光源更远，这意味着味道着场景中包含一个更靠近光源的对比图。
- 换句话说，当前片段在阴影中。



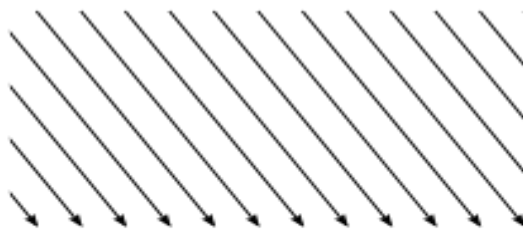
渲染阴影贴图

➤ 光源

- 只考虑**定向光**——距离太远以至所有光线都可以被认为是平稳的光。
- 渲染阴影贴图是使用**正交投影阵**完成的。
- 正交矩阵就像通常的透视投影阵一样，只是不考虑透视 - 一个物体无论远近看起来都是一样的。

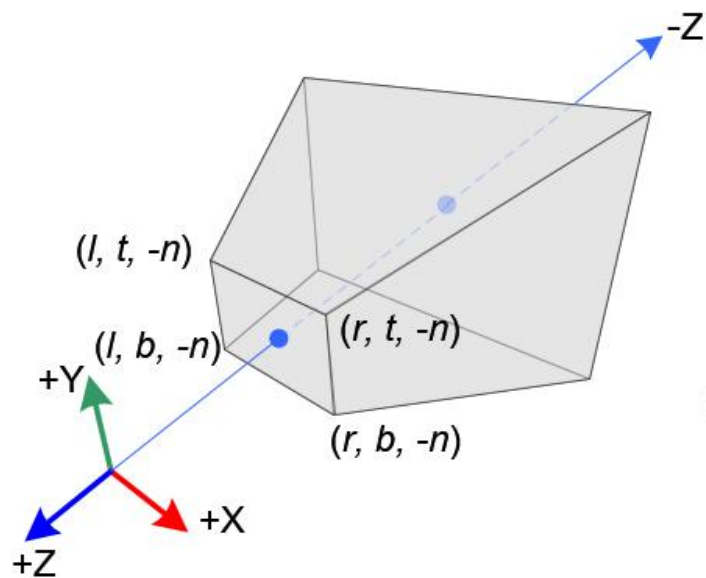


POINT LIGHT
emits light in
all directions.

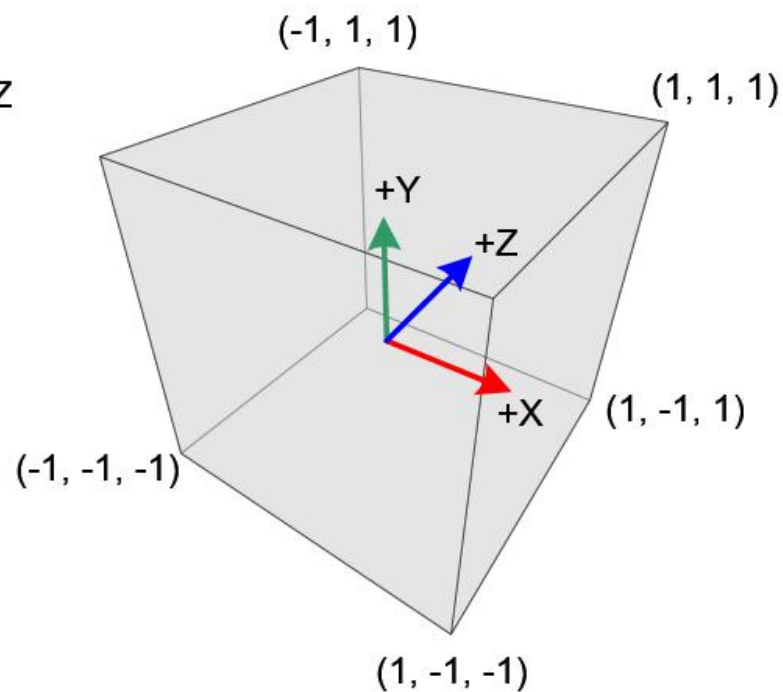


DIRECTIONAL LIGHT
has parallel light rays, all
from the same direction.

渲染阴影贴图



透视投影



正交投影

渲染阴影贴图

➤ 设置渲染目标 - sendDataToOpenGL ()

- 使用FrameBuffer

```
GLuint shadowFramebuffer = 0;  
glGenFramebuffers(1, & shadowFramebuffer);
```

- 创造深度纹理

```
GLuint depthTexture;  
glGenTextures(1, &depthTexture);  
glBindTexture(GL_TEXTURE_2D, depthTexture);  
glTexImage2D(GL_TEXTURE_2D, 0, GL_DEPTH_COMPONENT, 1024, 1024,  
0, GL_DEPTH_COMPONENT, GL_FLOAT, NULL);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);  
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
```


渲染阴影贴图

➤ 设置渲染目标 - sendDataToOpenGL ()

- 将 “ renderedTexture ” 设置为我们的颜色 附件#0

```
glBindFramebuffer(GL_FRAMEBUFFER, shadowFrameBuffer);  
glFramebufferTexture(GL_FRAMEBUFFER, GL_DEPTH_ATTACHMENT, depthTexture, 0);  
glDrawBuffer(GL_NONE); // No color buffer is drawn to.
```

- 开始最终检查我们的障碍物是否正常

```
if(glCheckFramebufferStatus(GL_FRAMEBUFFER) != GL_FRAMEBUFFER_COMPLETE)  
{cout << "FrameBuffer does not complete!" << endl;}
```

渲染阴影贴图

➤ 安装新的阴影贴图着色器

- 创建类似于installShaders () 的函数

```
void installShadowMapShaders(){
    ...
    string temp = readShaderCode("ShadowVertexShader.glsl");
    //new vertex shader file, should also create new shader file for fragment.
    adapter[0] = temp.c_str();
    glShaderSource(vertexShaderID, 1, adapter, 0);
    ...
    shadowMapProgramID = glCreateProgram(); //new program ID
    glAttachShader(shadowMapProgramID, vertexShaderID);
    glAttachShader(shadowMapProgramID, fragmentShaderID);
    glLinkProgram(shadowMapProgramID);
    if (!checkProgramStatus(shadowMapProgramID))
        return;
    ...
}
```

- 就像使用多个着色器一样，在下一个教程中详细介绍

渲染阴影贴图

➤ 渲染阴影贴图——paintGL ()

- 使用阴影贴图着色器

```
glUseProgram(shadowMapProgramID);  
glBindFramebuffer(GL_FRAMEBUFFER, shadowFramebuffer);  
glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);  
glViewport(0, 0, 1024, 1024);  
drawScene();
```

渲染阴影贴图

➤ 渲染阴影贴图——paintGL ()

- 创造用于从光的角度冲洗场景的深度 MVP 矩阵

```
glm::vec3 lightInvDir = glm::vec3(0.5f,2,2);
```

→ 光点

```
glm::mat4 depthProjectionMatrix =  
glm::ortho<float>(-10,10,-10,10,-10,20);
```

→ 投影矩阵是一个正交矩阵，它将包含X、Y和Z轴上轴对齐架(-10,10)、(-10,10)、(-10,20)中的所有内容。这些值是为了了。让我们的整个场面始终可见

```
glm::mat4 depthViewMatrix =  
glm::lookAt(lightInvDir, glm::vec3(0,0,0),  
glm::vec3(0,1,0));
```

→ 看 在 起源

```
glm::mat4 depthModelMatrix = glm::mat4(1.0);
```

→ 模型阵（转换阵）：
取决于不同的模型

渲染阴影贴图

➤ 渲染阴影贴图——paintGL ()

- 创建深度 MVP 矩阵

```
glm::mat4 depthMVP = depthProjectionMatrix * depthViewMatrix * depthModelMatrix ;
```

- 将不同模型的不同变换发送到当前绑定的着色器，
（阴影贴图着色器）和渲染阴影贴图

```
GLuint depthMatrixLocation = glGetUniformLocation ( shadowMapProgramID , "
depthMVP ");
glBindVertexArray ( penguinObj ) ; //绑定模型信息
glUniformMatrix4fv( depthMatrixLocation , 1, GL_FALSE, & depthMVP [0][0])
//将其转换发送到当前绑定的着色器
glDrawElements (GL_TRIANGLES, penguinObj.indices.size (),
GL_UNSIGNED_INT, 0); //渲染阴影贴图
```

渲染阴影贴图

- 新的阴影贴图顶部着色器
 - 计算顶部在齐次坐标中的位置

```
// Input vertex data, different for all executions of this shader.  
in layout(location = 0) vec3 position_modelSpace;  
  
// Values that stay constant for the whole mesh.  
uniform mat4 depthMVP;  
  
void main(){  
    gl_Position = depthMVP * vec4(position_modelSpace,1);  
}
```

渲染阴影贴图

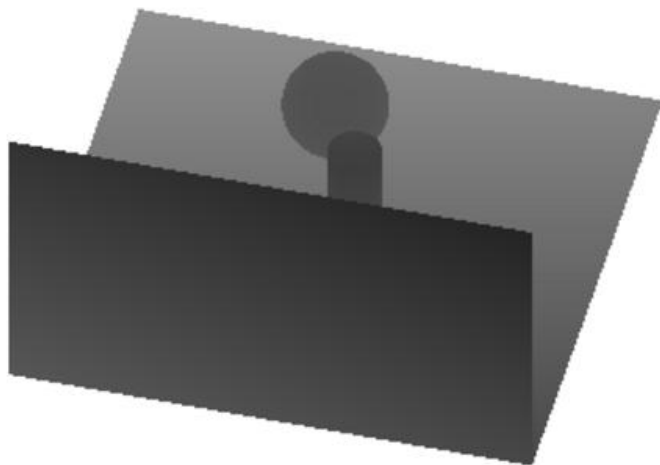
- 新的阴影贴图片段着色器
 - 在位置0处写入片段的深度（即在我们的深度纹理中）

```
// Output data
out layout(location = 0) float fragmentdepth;

void main(){
    // Not really needed, OpenGL does it anyway
    fragmentdepth = gl_FragCoord.z;
}
```

渲染阴影贴图

➤ 结果



深色表示小 z ；因此，墙的右上角靠近相机。相对，白色表示 $z=1$ （在齐次坐标中），所以这么远。

使用阴影贴图

➤ 基本着色器

- 对于我们计算的每个片段，我们必须测试它是否在阴影贴图“后面”。
- 为此，我们需要计算当片前段在制作阴影贴图时使用的同一个空间中的位置。
- 所以我们需要用通常的MVP阵变换一次，再用depthMVP阵变换一次。

```
//使用基本着色器
glUseProgram(programID);
glBindFramebuffer (GL_FRAMEBUFFER, 0 );
glClear (GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
glViewport (0, 0, 512, 512);
//设置经常使用的MVP阵型和光照信息
.....
//设置深度MVP阵营
.....
```

使用阴影贴图

➤ 基础着色器-关于深度MVP的小技巧

- 将顶部的位置乘以depthMVP将得到齐次坐标，它在 $[-1,1]$ 中；
- 但纹理采集必须在 $[0,1]$ 内完成。
- 示例：屏幕中间的片段将在 $(0,0)$ 齐次坐标中；但由于它必须对纹理的中间进行采样，因此UV必须为 $(0.5, 0.5)$ 。

➤ 解决

- 将齐次坐标以下面阵

```
glm::mat4 biasMatrix(  
    0.5, 0.0, 0.0, 0.0,  
    0.0, 0.5, 0.0, 0.0,  
    0.0, 0.0, 0.5, 0.0,  
    0.5, 0.5, 0.5, 1.0  
);  
glm::mat4 depthBiasMVP = biasMatrix * depthMVP ;
```

将坐标除以2（对角线： $[-1,1]$
→ $[-0.5, 0.5]$ ）
翻译它们（下排： $[-0.5, 0.5]$ -
→ $[0,1]$ ）

使用阴影贴图

➤ 基础着色器- 渲染模型

- 以“企鹅”模型为例

着色器中设置纹理位置：阴影贴图中使用的对象纹理和深度纹理

```
GLuint TextureID = glGetUniformLocation ( programID , " objTexture ");  
GLuint shadowMapID = glGetUniformLocation ( programID , " shadowMap ");
```

```
glBindVertexArray ( penguinObj ); //绑定模型信息
```

//绑定图像和深度纹理

```
glActiveTexture ( GL_TEXTURE0 );  
glBindTexture (GL_TEXTURE_2D,  
penguin_texture );  
glUniform1i( TextureID , 0 );
```

```
glActiveTexture ( GL_TEXTURE1 );  
glBindTexture (GL_TEXTURE_2D, depthTexture );  
glUniform1i( shadowMapID , 1 );
```

→ //已经加载到sendDataToOpenGL ()
penguin_texture = loadTexture (“penguin_01.jpg”);

→ sendDataToOpenGL ()和shadow FragmentShader
glFramebufferTexture(GL_FRAMEBUFFER,
GL_DEPTH_ATTACHMENT, depthTexture , 0);

使用阴影贴图

- 基础着色器- 渲染模型
- 以“企鹅”模型为例

```
//发送通常的MVP和深度MVP给着色器  
.....  
//绘制模型  
glDrawElements (...);
```

使用阴影贴图

- 顶部着色器
- 输入2个位置
 - `gl_Position`是从当前相机看到的顶部的位置
 - `ShadowCoord`是从光源看到的顶部位置

```
//从opengl获得MVP和
.....
// 顶部的输出位置，在裁剪空间中： MVP * position
gl_Position = MVP * vec4(vertexPosition_modelspace,1);

// 相同，但使用了光的图像矩阵
ShadowCoord = DepthBiasMVP * vec4( vertexPosition_modelspace , 1);
```

使用阴影贴图

➤ 片段着色器

```
//get ShadowCoord from vertexshader  
in vec4 shadow_vertexPosition;  
  
//get depth texture (calculated shadow map) from opengl  
uniform sampler2D shadowMap;
```

使用阴影贴图

- 片段着色器
- 如果当前片段比最近的遮挡物更远，这意味着我们在（最近的遮挡物的）阴影中
 - `texture( shadowMap , ShadowCoord.xy ).z` 是光源和最近的遮挡物之间的距离
 - `ShadowCoord.z`是光源和当前片段之间的距离

```
float visibility = 1.0;
if ( texture( shadowMap, ShadowCoord.xy ).z < ShadowCoord.z){
    visibility = 0.5; //hyperparameter depends on your models
}
```

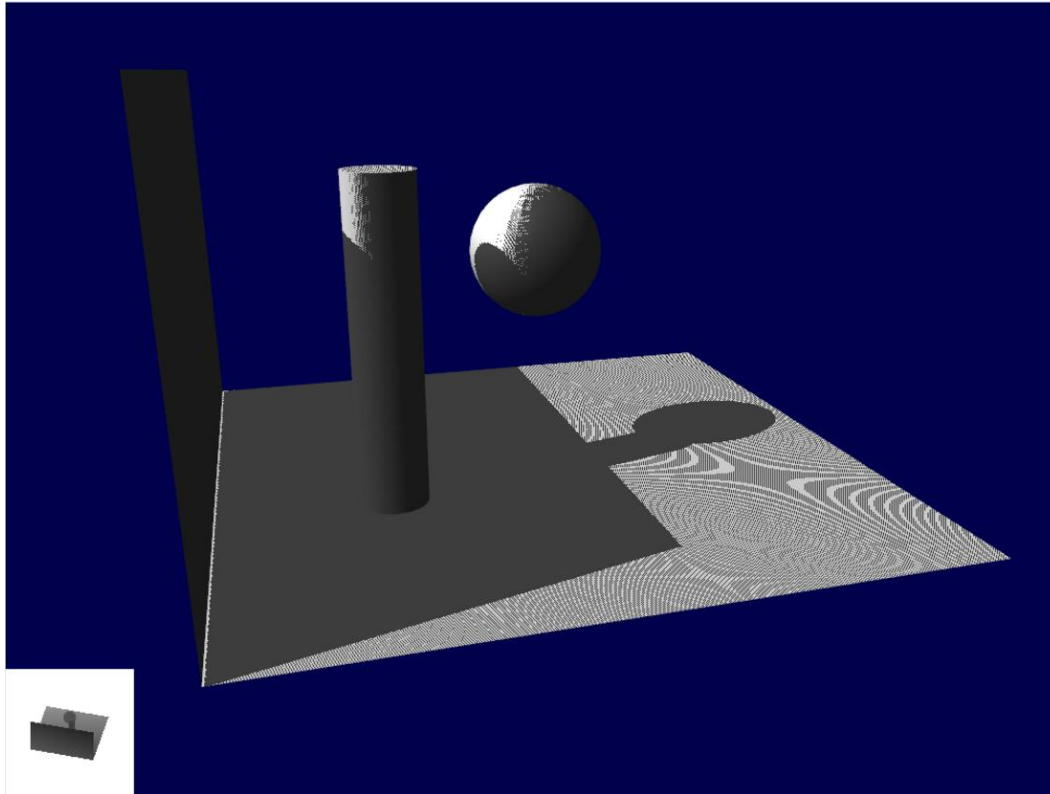
使用阴影贴图

- 片段着色器
- 修改着色

```
color =  
// Ambient : shadow mapping has no effect on ambient light  
MaterialAmbientColor * AmbientLightColor +  
// Diffuse and specular: use 'visibility' to control  
// Can set different visibilities to diffuse and specular light  
visibility * MaterialDiffuseColor * DiffuseLightColor * DiffuseBrightness +  
visibility * MaterialSpecularColor * SpecularLightColor * pow(SpecularBrightness,50);
```

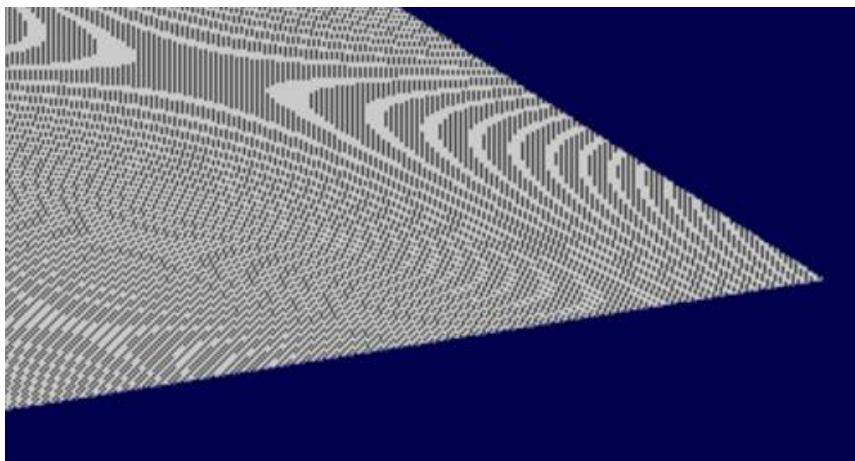

使用阴影贴图

- 结果
- 全局意识它在那里，但质量是不可接受的。



Shadow acne

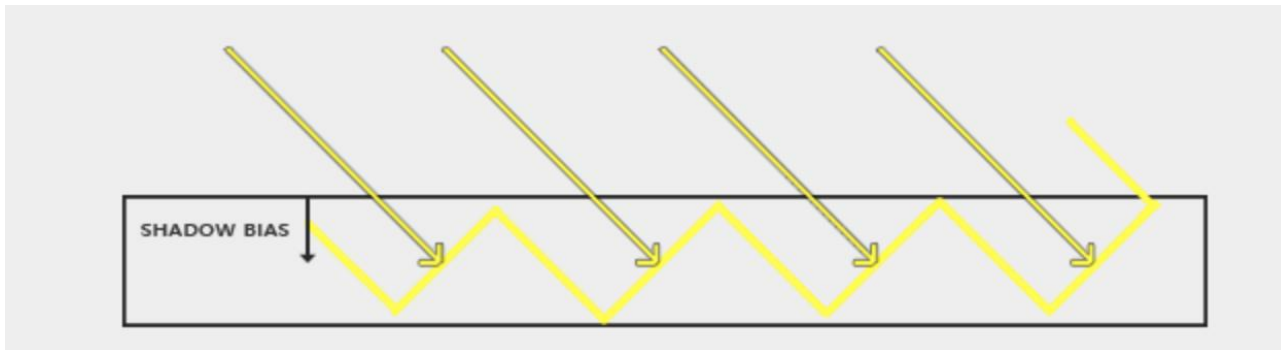
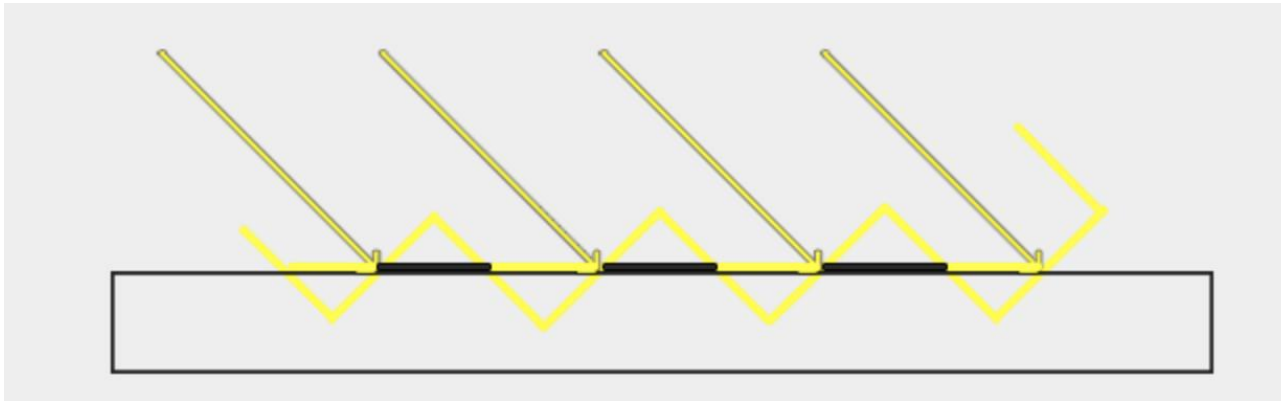
- 最明显的问题是Shadow acne



- 解决：添加错误偏差范围（偏差）
- 如果 当前片段的深度（相同，在光照空间中）**确实**远大于光照贴图值。

```
float bias = 0.005; //hyperparameter depends on your models
if ( texture( shadowMap, ShadowCoord.xy ).z > ShadowCoord.z + bias ){
    return shadow; //set it as shadow
}
```

Shadow acne: far away fragments



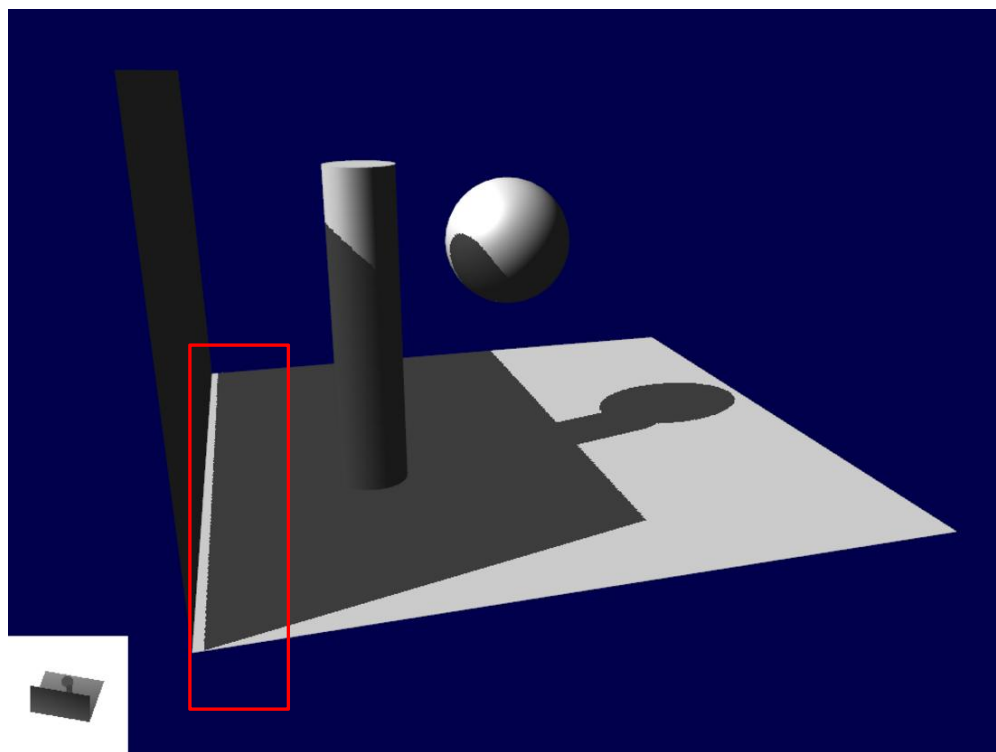
Shadow acne

- 更多的 坚硬的方法 为了了 陡 角度 到 这个 光 来源： 根据 倾斜率修改偏差

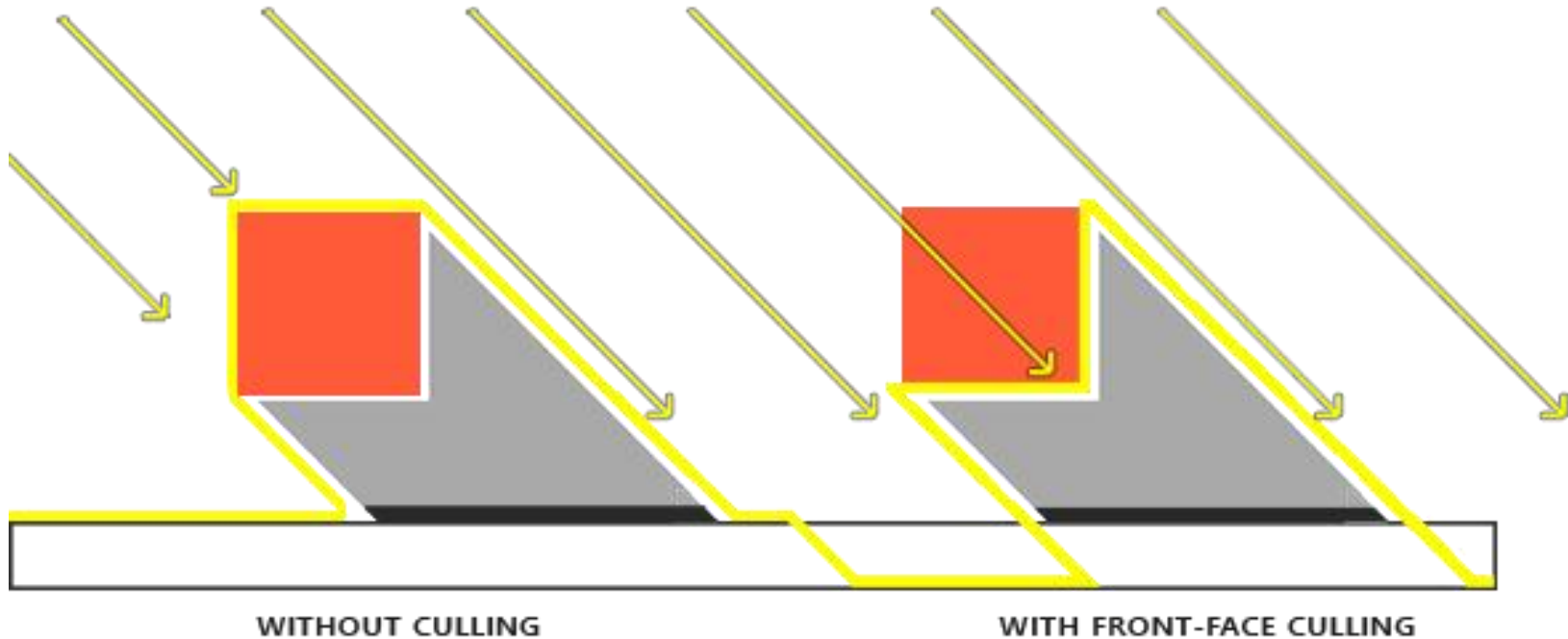
```
float bias = max(0.05 * (1.0 - dot(normal, lightDir)), 0.005);
```

Peter Panning

错误的地面阴影，使墙看起来像在飞翔（因此得名“Peter Panning”），添加 偏见 使它更差



Peter Panning

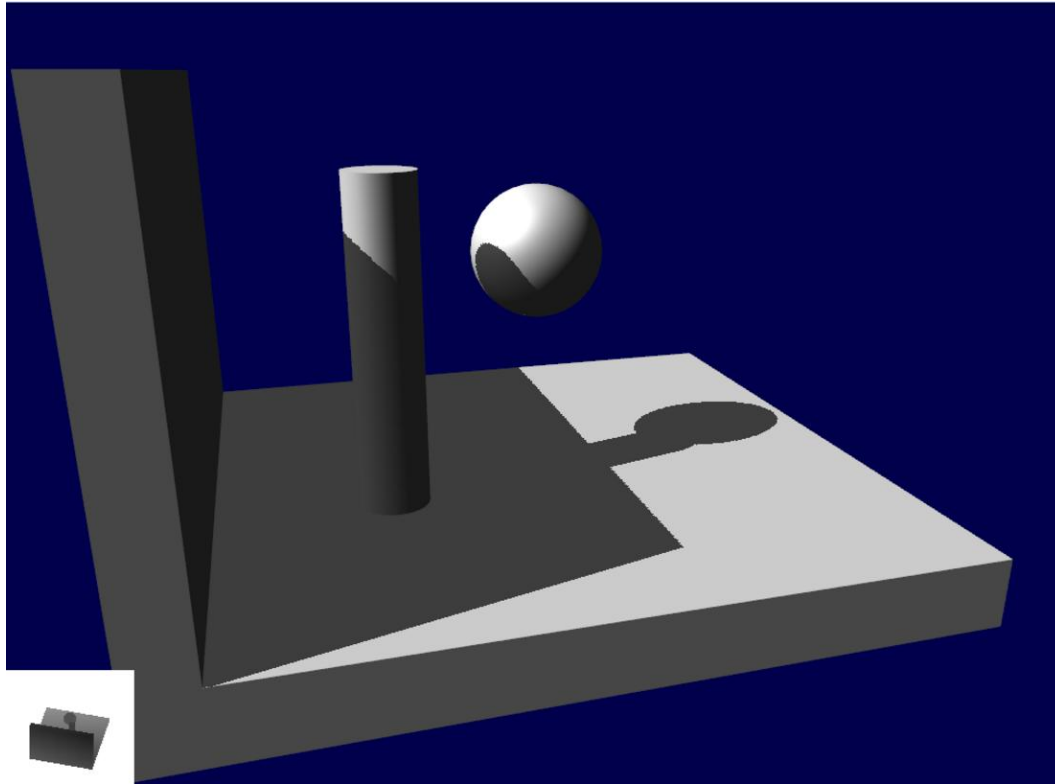


```
glCullFace (GL_FRONT);  
RenderSceneToDepthMap ();  
glCullFace (GL_BACK); // 不要忘记重新设置原来的删除面
```

Peter Panning

简单地避免薄几何

➤ 结果:



其他 问题： 别名

其他 技巧： PCF

你 能 参 考 到：

<https://learnopengl.com/Advanced-Lighting/Shadows/Shadow-Mapping>